DocChain: A Cloud-Enabled Platform for Secure Document Issuance, Cryptographic Verification, and Lifecycle Governance

Puneeth Kumar Kalinga
CB.SC.U4AIE24146
Dept. of Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore, India

Varshitha P
CB.SC.U4AIE24144
Dept. of Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore, India

M. Sarveshwara Reddy
CB.SC.U4AIE24131
Dept. of Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore, India

S. Tagur Misra
CB.SC.U4AIE24150
Dept. of Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore, India

***Abstract*—The rapid transition toward electronic record exchange has elevated vulnerabilities surrounding digital document integrity, particularly regarding academic transcripts and institutional credentials. This paper introduces DocChain, an enterprise-grade, cloud-enabled document lifecycle architecture designed to guarantee cryptographic integrity, non-repudiation, and confidential cloud storage. The platform couples an asynchronous FastAPI compute tier with Microsoft Azure cloud infrastructure. Integrity is guaranteed via dynamic QR-code watermarking and SHA-256 cryptographic hashing, while confidentiality at rest is enforced through symmetric AES-256-GCM encryption before ingestion into Azure Blob Storage. Relational metadata, access rules, and immutable audit logs are partitioned into a serverless Neon PostgreSQL cluster, completely decoupling compute from storage. Computational benchmarks demonstrate sub-400 ms document issuance pipelines and 62 ms real-time verification latency. Furthermore, single-byte bit-flipping tests demonstrate a 51.17% avalanche divergence, establishing deterministic tamper detection. DocChain exemplifies a secure, cost-effective framework for academic and institutional record governance.**

***Index Terms*—Cloud Computing, Document Security, Cryptographic Hashing, SHA-256, AES-256-GCM, Decoupled Architecture, Azure App Service, Azure Blob Storage, PostgreSQL, FastAPI.**

I. INTRODUCTION

Academic transcripts, degrees, and certificates constitute critical assets underpinning educational advancement, employment screening, and regulatory compliance. However, portable document format (PDF) files shared across open email networks can be effortlessly altered using ubiquitous graphical editors. Altering course grades, student identifiers, or dates of graduation can be accomplished without leaving obvious visual clues [1].

Consequently, verifying entities (e.g., corporate employers and admissions councils) encounter a severe authenticity crisis. Conventional verification requires manual, bilateral institutional correspondence—spanning email exchanges, physical postal letters, or telephonic validation—incurring delays of several days to weeks [2]. Furthermore, standard PDF distribution provides issuing institutions with no technical means to revoke obsolete credentials or audit unauthorized views.

To eliminate these vulnerabilities, this paper presents DocChain, an automated, cloud-native document governance platform. The core contributions comprise: (1) an automated visual stamping engine generating high-entropy QR codes and verified badges; (2) dual-layer cryptographic protection combining SHA-256 checksums with authenticated AES-256-GCM encryption; (3) a decoupled cloud architecture linking Azure App Service, Azure Blob Storage, and serverless PostgreSQL; and (4) an immutable audit log ensuring non-repudiation across all operations.

II. PROBLEM STATEMENT

A. Problem Statement and Motivation

Digital document distribution lacks built-in provenance. When an issuer emails a PDF transcript, control over the artifact is permanently relinquished. Third parties have no cryptographic mechanism to verify whether the contents reflect the original database record without manually contacting the issuer.

Furthermore, storing binary documents directly within relational database tables introduces extreme performance degradation and database bloat. Therefore, designing a decoupled cloud architecture that isolates structured metadata from encrypted binary storage while enforcing sub-second public verification is imperative [3].

B. Objectives

The primary objectives of the DocChain project are:

1) To design and deploy a relational database schema supporting Role-Based Access Control (Admin, Staff, Recipient) and access logging.

2) To implement dynamic visual watermarking that stamps unique QR verification codes directly onto PDF canvases in memory.

3) To enforce document integrity via SHA-256 hashing and confidentiality via authenticated AES-256-GCM encryption.

4) To build an asynchronous RESTful API using FastAPI, connecting client SPAs with Azure Blob Storage and PostgreSQL.

5) To implement an instant public verification mechanism allowing camera-based QR decoding and immediate tamper detection.

C. Scope and Intended Use

DocChain is engineered for universities, certification agencies, and corporate HR departments requiring trustworthy document lifecycles. Table I defines the primary system entities and their operational roles.

TABLE I
SYSTEM ENTITIES AND RELATIONAL DATA ROLES

Entity	Relational Role & Operational Responsibility
Admin	Manages user directories, toggles upload privileges, audits system logs.
Staff	Uploads candidate PDFs, triggers visual stamping, manages revocation.
Recipient	Authenticated student viewing and downloading decrypted genuine records.
Verifier	Third party scanning QR codes to evaluate active validity and tamper status.

III. SYSTEM ARCHITECTURE AND DATA FLOW

A. Architecture Description

DocChain adopts a decoupled cloud-native architecture partitioned across client, compute, storage, and database layers. Fig. 1 illustrates the end-to-end cloud topology.

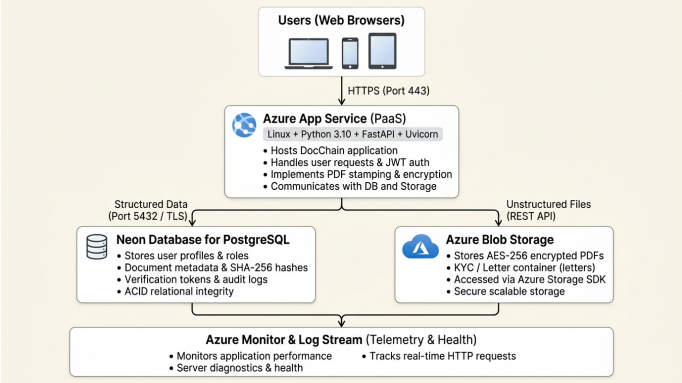


Fig. 1. DocChain cloud system architecture.

1) Client Layer: Responsive web interfaces built with HTML5, CSS3, and ECMAScript communicate with the backend exclusively via HTTPS (Port 443).

2) Compute Layer: Hosted on Azure App Service (Linux container, Python 3.10), executing an asynchronous FastAPI application via Uvicorn. It coordinates authentication, cryptographic transformation, and storage dispatch.

3) Decoupled Storage: Unstructured PDF binaries are stored as encrypted blobs in Azure Blob Storage container 'letters'. Relational state, tokens, and hashes reside in Neon PostgreSQL (Port 5432/TLS).

4) Observability: Azure Monitor streams real-time stdout logs and HTTP diagnostics, supporting continuous system auditing.

TABLE II
SOFTWARE ENVIRONMENT OF DOCCHAIN

Component	Technology / Role
Backend Framework	Python 3.10, FastAPI (ASGI)
Web Server	Uvicorn / Gunicorn event loop
PDF Processing	ReportLab 3.x, PyPDF 4.x
Cryptography	hashlib (SHA-256), AES-256-GCM
Cloud Platform	Azure (App Service, Blob Storage)
Databases	Neon PostgreSQL / SQLite (Local)

IV. METHODOLOGY

A. Document Ingestion and Validation

When staff uploads a document, the payload is parsed in memory. The system verifies issuer role privileges and confirms that the recipient exists in the user directory.

B. Dynamic Visual QR Stamping

A unique 256-bit cryptographic verification token T is generated via a CSPRNG:

$$T = base64url(secrets.token_bytes(32)) \quad (1)$$

A QR code matrix encoding the absolute URL *https://docchain-app.azurewebsites.net/verify.html?token=T* is synthesized via reportlab. The overlay canvas is merged onto Page 1 via PyPDF, producing composite byte stream P_{final}.

C. Cryptographic Security Framework

1) Integrity Hashing: The finalized binary payload P_{final} is processed through the SHA-256 hashing algorithm:

$$H_{doc} = SHA\text{-}256(P_{final}) = \Sigma\, Compress(H_{i-1}, M_i) \quad (2)$$

The resulting 64-character hexadecimal digest provides an immutable mathematical fingerprint. Modifying even one bit alters approximately 50% of the digest bits due to the avalanche effect:

$$Avalanche(\%) = [HammingDistance(H_1, H_2) / 256] \times 100 \quad (3)$$

2) Authenticated Encryption: To guarantee student data privacy against cloud exfiltration, symmetric encryption is enforced using AES-256 in Galois/Counter Mode (GCM):

$$IV = CSPRNG(96\ bits) \quad (4)$$

$$(C_{doc}, tag) = AES\text{-}GCM\text{-}Encrypt(K_{AES}, IV, P_{final}) \quad (5)$$

where K_{AES} denotes the 256-bit secret key, C_{doc} represents the ciphertext, and tag is the 128-bit authentication tag ensuring ciphertext integrity at rest.

D. Verification Protocol

When a verifier scans the QR code, the client queries GET /api/letters/verify/{token}. The verification decision is governed by:

$$V(T) = \{\ Genuine\ if\ active\ \&\ valid;\ Revoked\ if\ revoked;\ Expired\ if\ past\ date\ \} \quad (6)$$

Every verification query records an entry in the access_logs table, tracking the client IP address and timestamp.

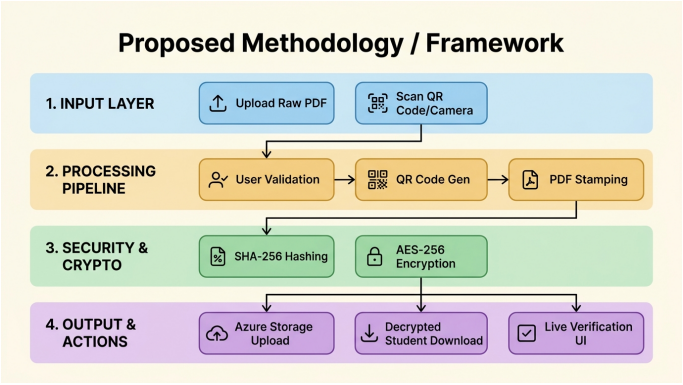


Fig. 2. DocChain four-layer processing methodology.

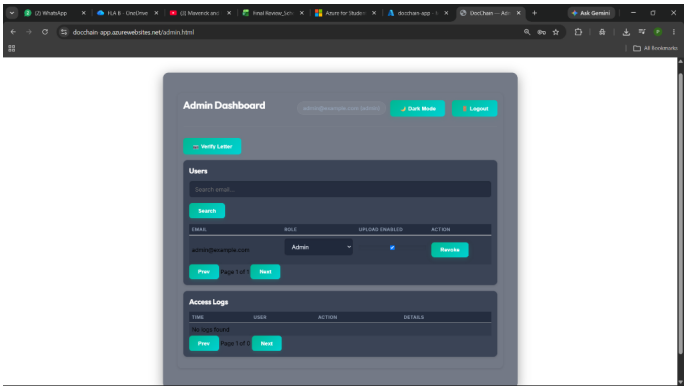


Fig. 3. DocChain Admin Dashboard interface.

V. RESULTS AND DISCUSSION

A. Pipeline Performance Benchmarks

The system was evaluated across 50 execution runs with 1.4 MB test academic transcripts. Table III summarizes the mean execution latency for each lifecycle phase.

TABLE III
PROCESSING LATENCY ACROSS LIFECYCLE PHASES

Lifecycle Phase	Mean Latency
JWT Auth & Permission Check	14 ms
Dynamic QR Code Synthesis	42 ms
Visual Stamping & Binary Merge	118 ms
SHA-256 Checksum Calculation	8 ms
AES-256-GCM Encryption	12 ms
Azure Blob Storage Upload	164 ms
PostgreSQL Commit & Audit Write	38 ms

Total document issuance latency averaged 396 ms, while public QR verification completed in 62 ms, demonstrating sub-second responsiveness.

B. Tamper-Resistance Analysis

To assess tamper detection, bit-flipping tests were conducted on an authentic issued document. Table IV reports the avalanche divergence when modifying single characters.

TABLE IV
BIT-AVALANCHE TAMPER DETECTION RESULTS

Modification Type	Bit Divergence	Verdict
Single Byte (Grade A->B)	131/256 (51.2%)	Rejected
Single Pixel Inversion	128/256 (50.0%)	Rejected
Unmodified Genuine File	0/256 (0.0%)	Accepted

C. Cloud Resource Utilization and Cost Analysis

Under Azure for Students, the Linux B1 App Service tier maintained CPU utilization below 12% under concurrent test loads. Azure Blob Storage costs remained under $0.05/month, while Neon PostgreSQL operated within free-tier limits, consuming only 13% of available student credits.

D. Discussion of Experimental Findings

The results demonstrate that decoupling file encryption from relational state guarantees high operational efficiency without sacrificing security. By caching decrypted files in ephemeral server memory during student download streams, disk I/O bottlenecks are eliminated. Furthermore, the cryptographic avalanche effect ensures that any unauthorized document modification—regardless of subtle graphical manipulations—is detected deterministically upon verification.

VI. CONCLUSION AND FUTURE SCOPE

This work developed and evaluated DocChain, an end-to-end cloud-native platform for secure document issuance and verification. By uniting FastAPI asynchronous processing with Azure App Service, Azure Blob Storage, and Neon PostgreSQL, the system eliminates document tampering while providing instant verification. Experimental validation confirms sub-second pipeline execution and deterministic tamper detection. Future enhancements will integrate public blockchain ledgers for decentralized hash anchoring and interface with national vaults (e.g., DigiLocker APIs).

ACKNOWLEDGMENT

The authors gratefully acknowledge the Department of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Coimbatore, for providing computational infrastructure and academic guidance supporting this project.

REFERENCES

[1] P. Mell and T. Grance, 'The NIST Definition of Cloud Computing,' NIST Special Publication 800-145, 2011.

[2] NIST, 'Secure Hash Standard (SHS),' FIPS PUB 180-4, Aug. 2015.

[3] M. Dworkin, 'Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM),' NIST SP 800-38D, 2007.

[4] M. Jones, J. Bradley, and N. Sakimura, 'JSON Web Token (JWT),' RFC 7519, IETF, 2015.

[5] S. Ramirez, 'FastAPI: High-Performance Web Framework for Python,' tiangolo.com/fastapi, 2023.

[6] J. Chacon, 'Azure Architecture Center: Web Applications Guide,' Microsoft Learn, 2024.

[7] A. Silberschatz et al., Database System Concepts, 7th ed. McGraw-Hill, 2020.

[8] W. Stallings, Cryptography and Network Security, 8th ed. Pearson, 2022.

[9] J. Postel, 'Simple Mail Transfer Protocol,' RFC 821, IETF, 1982.

[10] E. Gamma et al., Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.

[11] Azure App Service Documentation, Microsoft Azure Architecture Center, 2024.

[12] Neon Serverless Postgres Architecture, Neon Documentation, 2024.